# Q21. What is the Collection Framework in Java?

## Simple Answer

The **Collection Framework** is a set of classes and interfaces provided by Java to store, manage, and manipulate groups of objects efficiently.

It provides ready-made data structures like **List, Set, Queue, and Map**, along with algorithms for searching, sorting, and iteration.

## Detailed Explanation

Before the Collection Framework, developers managed data using arrays, which have a fixed size.

The Collection Framework provides:

- Dynamic resizing
- Better performance
- Reusable data structures
- Standard APIs

The core interfaces are:

- List
- Set
- Queue

**Note:** `Map` is part of the Java Collections Framework but does **not** extend the `Collection` interface.

## Real-Time Example

An e-commerce application stores:

- Customer List → List
- Unique Product Categories → Set
- Order Processing → Queue
- Product ID → Product Details → Map

Different collection types solve different business problems.

## Code Example

```
import java.util.*;

public class Main {
```

```
    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Murali");
        names.add("John");

        System.out.println(names);
    }
}
```

**Output**

```
[Murali, John]
```

# Interview Tip

A very common interview question is:

**"Is Map part of the Collection Framework?"**

Answer:

**Yes**, it belongs to the Collection Framework, but it **does not implement the Collection interface**.

# Key Points

✓ Provides reusable data structures.

✓ Supports dynamic memory allocation.

✓ Improves performance and maintainability.

✓ Includes List, Set, Queue, and Map.

# Q22. What is the difference between List, Set, Queue, and Map?

# Simple Answer

| Interface | Duplicate Elements | Ordering | Null Allowed |
|---|---|---|---|

| | | | |
|---|---|---|---|
| List | ✅ Yes | Maintains insertion order | ✅ Yes |
| Set | ❌ No | Depends on implementation | Usually one null (HashSet) |
| Queue | Depends | FIFO | Depends on implementation |
| Map | Duplicate keys ❌, Duplicate values ✅ | Depends on implementation | One null key (HashMap), multiple null values |

# Detailed Explanation

## List

- Ordered collection.
- Allows duplicates.
- Index-based access.

Examples:

- ArrayList
- LinkedList

## Set

- Stores unique elements.
- No duplicates.

Examples:

- HashSet
- LinkedHashSet
- TreeSet

## Queue

- Follows FIFO (First In, First Out).

Examples:

- PriorityQueue
- ArrayDeque

## Map

Stores data as:

```
Key → Value
```

Examples:

- HashMap
- LinkedHashMap
- TreeMap

# Real-Time Example

Hospital Management System

- Patients admitted → List
- Unique Doctors → Set
- Waiting Queue → Queue
- Patient ID → Patient Record → Map

# Code Example

```
List<String> list = new ArrayList<>();
Set<String> set = new HashSet<>();
Queue<String> queue = new LinkedList<>();
Map<Integer, String> map = new HashMap<>();
```

# Interview Tip

Remember this simple rule:

- **List → Ordered**
- **Set → Unique**
- **Queue → FIFO**
- **Map → Key-Value Pair**

# Key Points

✓ List maintains insertion order.

✓ Set removes duplicates.

✓ Queue processes elements sequentially.

✓ Map stores key-value pairs.

# Q23. What is the difference between Array and ArrayList?

# Simple Answer

An **Array** has a fixed size and can store primitive data types as well as objects.

An **ArrayList** is a resizable collection that stores only objects and provides many built-in methods.

# Detailed Explanation

| Array | ArrayList |
|---|---|
| Fixed size | Dynamic size |
| Can store primitives | Stores objects (wrapper classes for primitives) |
| Faster for fixed-size data | More flexible |
| No built-in methods | Rich API (`add()`, `remove()`, `contains()`) |

# Real-Time Example

Suppose your application stores the months of the year.

Use an **Array** because the size is fixed (12).

If you're storing users who register dynamically, use an **ArrayList** because the number of users changes.

# Code Example

```
int[] numbers = {10, 20, 30};

ArrayList<Integer> list = new ArrayList<>();

list.add(10);
list.add(20);
list.add(30);

System.out.println(list);
```

# Interview Tip

If the number of elements can change during execution, choose **ArrayList** over an array.

# Key Points

✓ Array → Fixed size.

✓ ArrayList → Dynamic size.

✓ ArrayList provides many utility methods.

✓ Arrays can store primitives directly.

# Q24. What is ArrayList? What are its advantages and disadvantages?

# Simple Answer

**ArrayList** is a resizable implementation of the `List` interface.

It maintains insertion order, allows duplicate elements, and provides fast random access using indexes.

# Detailed Explanation

### Advantages

- Dynamic size
- Fast retrieval using indexes
- Easy insertion at the end
- Rich API support

### Disadvantages

- Slow insertion/deletion in the middle because elements must be shifted.
- Not synchronized.

# Real-Time Example

An online shopping application stores recently viewed products.

Products are displayed in the order viewed, duplicates are allowed, and random access is common.

ArrayList is a suitable choice.

# Code Example

```
ArrayList<String> products = new ArrayList<>();

products.add("Laptop");
products.add("Mobile");
products.add("Watch");

System.out.println(products.get(1));
```

**Output**

```
Mobile
```

# Interview Tip

Remember the time complexities:

- `get()` → **O(1)**
- `add()` at end → **O(1)** (amortized)
- Insert/Delete in middle → **O(n)**

Interviewers often ask these.

# Key Points

✓ Dynamic array implementation.

✓ Maintains insertion order.

✓ Allows duplicates.

✓ Best for frequent read operations.

# Q25. What is the difference between ArrayList and LinkedList?

# Simple Answer

Both **ArrayList** and **LinkedList** implement the `List` interface, but they use different internal data structures.

- **ArrayList** uses a dynamic array.
- **LinkedList** uses a doubly linked list.

# Detailed Explanation

| ArrayList | LinkedList |
|---|---|
| Dynamic array | Doubly linked list |
| Fast random access | Sequential access |
| Slow insert/delete in middle | Fast insert/delete once position is reached |
| Less memory | More memory (stores node links) |

### When to use?

Use **ArrayList** when:

- Reading data frequently.
- Random access is important.

Use **LinkedList** when:

- Frequent insertions and deletions occur, especially near the beginning or middle.

# Real-Time Example

### ArrayList

Displaying products in an online shopping website where users frequently access items by index.

### LinkedList

Music playlist where songs are frequently inserted or removed.

# Code Example

```
List<String> arrayList = new ArrayList<>();
arrayList.add("Java");

List<String> linkedList = new LinkedList<>();
linkedList.add("Spring");

System.out.println(arrayList);
System.out.println(linkedList);
```

# Interview Tip

One of the most common Java interview questions:

**"Which is faster: ArrayList or LinkedList?"**

Answer:

- **Reading** → ArrayList
- **Insertion/Deletion** → LinkedList (after reaching the node)

# Key Points

✓ ArrayList uses a dynamic array.

✓ LinkedList uses a doubly linked list.

✓ ArrayList is better for searching and reading.

✓ LinkedList is better for frequent insertions and deletions.

# ✅ Collections Set 1 Complete

**Covered Questions:**

21. What is the Collection Framework in Java?
22. Difference between List, Set, Queue, and Map
23. Difference between Array and ArrayList
24. What is ArrayList?
25. Difference between ArrayList and LinkedList

**Review**

This set establishes the foundation of the Collections Framework. The questions progress naturally from the overall framework to the most commonly used `List` implementations. These are among the highest-frequency questions in Java interviews for 0–2 years and provide the groundwork before moving on to `Set`, `Map`, `Queue`, iterators, and sorting in the next sets.

# Q26. What is the difference between HashSet, LinkedHashSet, and TreeSet?

## Simple Answer

`HashSet`, `LinkedHashSet`, and `TreeSet` are implementations of the **Set** interface. All of them store **unique elements**, but they differ in ordering, performance, and internal implementation.

# Detailed Explanation

| Feature | HashSet | LinkedHashSet | TreeSet |
|---|---|---|---|
| Duplicates | ❌ Not Allowed | ❌ Not Allowed | ❌ Not Allowed |
| Insertion Order | ❌ No | ✅ Yes | ❌ No (Sorted Order) |
| Sorting | ❌ No | ❌ No | ✅ Natural/Custom Order |
| Internal Structure | Hash Table | Hash Table + Linked List | Red-Black Tree |
| Performance | Fastest | Slightly Slower | Slower than HashSet |

### When to use?

- **HashSet** → Fast lookup and uniqueness.
- **LinkedHashSet** → Maintain insertion order.
- **TreeSet** → Automatically sort elements.

# Real-Time Example

### HashSet

Unique email IDs in a registration system.

### LinkedHashSet

Browser history while maintaining the order in which pages were visited.

### TreeSet

Student marks displayed in ascending order.

# Code Example

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Set<Integer> hashSet = new HashSet<>();
```

```
        hashSet.add(30);
        hashSet.add(10);
        hashSet.add(20);

        Set<Integer> linkedHashSet = new LinkedHashSet<>();
        linkedHashSet.add(30);
        linkedHashSet.add(10);
        linkedHashSet.add(20);

        Set<Integer> treeSet = new TreeSet<>();
        treeSet.add(30);
        treeSet.add(10);
        treeSet.add(20);

        System.out.println(hashSet);
        System.out.println(linkedHashSet);
        System.out.println(treeSet);
    }
}
```

**Possible Output**

```
[20, 10, 30]
[30, 10, 20]
[10, 20, 30]
```

# Interview Tip

One of the most common questions:

**Which Set should I use if I need sorted data?**

Answer:

**TreeSet**

# Key Points

✓ All Sets store unique elements.

✓ HashSet is the fastest.

✓ LinkedHashSet preserves insertion order.

✓ TreeSet automatically sorts elements.

# Q27. What is the difference between HashMap, LinkedHashMap, and TreeMap?

## Simple Answer

All three classes implement the **Map** interface and store data as **key-value pairs**.

They differ in ordering, sorting, and performance.

## Detailed Explanation

| Feature | HashMap | LinkedHashMap | TreeMap |
|---|---|---|---|
| Duplicate Keys | ❌ Not Allowed | ❌ Not Allowed | ❌ Not Allowed |
| Duplicate Values | ✅ Allowed | ✅ Allowed | ✅ Allowed |
| Insertion Order | ❌ No | ✅ Yes | ❌ Sorted by Key |
| Sorting | ❌ No | ❌ No | ✅ Yes |
| Null Key | ✅ One | ✅ One | ❌ No |
| Internal Structure | Hash Table | Hash Table + Linked List | Red-Black Tree |

### When to use?

- **HashMap** → Fast lookup.
- **LinkedHashMap** → Preserve insertion order.
- **TreeMap** → Store keys in sorted order.

## Real-Time Example

### HashMap

Employee ID → Employee Details

### LinkedHashMap

Recently viewed products.

### TreeMap

Dictionary application displaying words alphabetically.

# Code Example

```
Map<Integer, String> map = new HashMap<>();

map.put(101, "Murali");
map.put(102, "John");

System.out.println(map);
```

# Interview Tip

Frequently asked question:

**Why doesn't TreeMap allow a null key?**

Answer:

TreeMap sorts keys using comparison (`compareTo()` or `Comparator`), and `null` cannot be compared.

# Key Points

✓ HashMap → Fastest.

✓ LinkedHashMap → Maintains insertion order.

✓ TreeMap → Sorted keys.

✓ Keys are always unique.

# Q28. How does HashMap work internally?

# Simple Answer

A **HashMap** stores data using a **hash table**.

When a key-value pair is inserted:

1. The key's `hashCode()` is calculated.
2. The hash value determines the bucket.

3. If multiple keys map to the same bucket, HashMap handles the collision using a linked list or a balanced tree (Java 8+).

# Detailed Explanation

## Step 1

```
map.put(101, "Murali");
```

The key's `hashCode()` is calculated.

## Step 2

The hash code determines the bucket index.

## Step 3

If another key maps to the same bucket, HashMap stores both entries.

Since Java 8:

- Small collisions → Linked List
- Large collisions → Red-Black Tree

This improves lookup performance.

# Real-Time Example

Think of a hotel.

Room Number → Bucket

Guest → Value

Each room stores one or more guests (in case of collision).

# Code Example

```
HashMap<Integer, String> map = new HashMap<>();

map.put(101, "Murali");
map.put(102, "John");
```

```
System.out.println(map.get(101));
```

**Output**

```
Murali
```

# Interview Tip

Very frequently asked interview question:

**Why should we override both `equals()` and `hashCode()` when using custom objects as HashMap keys?**

Answer:

- `hashCode()` identifies the bucket.
- `equals()` identifies the correct object within that bucket.

Both must be consistent.

# Key Points

✓ Uses a hash table.

✓ Uses `hashCode()` and `equals()`.

✓ Handles collisions.

✓ Java 8 uses Red-Black Trees for heavy collisions.

# Q29. What is the difference between Comparable and Comparator?

# Simple Answer

Both **Comparable** and **Comparator** are used for sorting objects.

- **Comparable** defines the natural ordering of objects.
- **Comparator** defines a custom ordering.

# Detailed Explanation

| Comparable | Comparator |
| --- | --- |

| Package: `java.lang` | Package: `java.util` |
| --- | --- |
| Uses `compareTo()` | Uses `compare()` |
| Natural sorting | Custom sorting |
| Changes the class | Separate class or lambda |

### When to use?

Use **Comparable** when there is only one natural sorting order.

Use **Comparator** when multiple sorting criteria are needed.

# Real-Time Example

Employee sorting:

Natural order → Employee ID

Custom order:

- Salary
- Name
- Age

Different Comparators can be created for each requirement.

# Code Example

```
class Employee implements Comparable<Employee> {

    int id;

    Employee(int id) {
        this.id = id;
    }

    @Override
    public int compareTo(Employee e) {
        return this.id - e.id;
    }
}
```

# Interview Tip

Common interview question:

**Which one is better?**

Answer:

Comparable for default sorting.

Comparator for multiple sorting requirements.

# Key Points

✓ Comparable → Natural ordering.

✓ Comparator → Custom ordering.

✓ Comparator is more flexible.

✓ Frequently used with `Collections.sort()` and Streams.

# Q31. What is the difference between `Collection` and `Collections` in Java?

# Simple Answer

`Collection` is an **interface** that represents a group of objects, whereas `Collections` is a **utility class** that provides static methods to perform operations such as sorting, searching, and reversing collections.

# Detailed Explanation

## Collection

- Root interface of the Collection Framework.
- Extended by:
  - List
  - Set
  - Queue

Examples:

- ArrayList

- LinkedList
- HashSet

### Collections

A utility class that provides helper methods such as:

- `sort()`
- `reverse()`
- `shuffle()`
- `binarySearch()`
- `max()`
- `min()`
- `frequency()`

# Real-Time Example

Suppose you have a list of employee names.

- **Collection** stores the names.
- **Collections.sort()** sorts them alphabetically.

# Code Example

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("John");
        names.add("Murali");
        names.add("Alex");

        Collections.sort(names);

        System.out.println(names);
    }
}
```

**Output**

```
[Alex, John, Murali]
```

# Interview Tip

This is one of the easiest interview questions, but many candidates confuse the two.

Remember:

**Collection = Interface**

**Collections = Utility Class**

# Key Points

✓ Collection is an interface.

✓ Collections is a utility class.

✓ Collections contains static helper methods.

✓ Collection stores data.

# Q35. Which Collection should you choose in different real-world scenarios?

# Simple Answer

The choice of collection depends on the business requirement.

There is no single "best" collection.

# Detailed Explanation

| Requirement | Best Collection |
|---|---|
| Maintain insertion order | ArrayList / LinkedHashSet / LinkedHashMap |
| Unique values | HashSet |
| Sorted data | TreeSet / TreeMap |
| Fast lookup using key | HashMap |
| FIFO processing | Queue |

| | |
|---|---|
| Frequent insert/delete | LinkedList |
| Thread-safe collection | CopyOnWriteArrayList / ConcurrentHashMap |

# Real-Time Example

## Shopping Website

Products

→ ArrayList

## User Login

Unique usernames

→ HashSet

## Employee Database

Employee ID → Employee Details

→ HashMap

## Ticket Booking Queue

Customers waiting

→ Queue

## Leaderboard

Scores sorted automatically

→ TreeSet

# Code Example

```
Map<Integer, String> employees = new HashMap<>();
```

```
employees.put(101, "Murali");
employees.put(102, "John");

System.out.println(employees.get(101));
```